

# **PhishGuard: A Multi-Layered Cyber Threat Intelligence Framework for Real-Time Phishing URL Detection**

SUBJECT CODE : 24CS634

Submitted by  
BHAVANA POLI - BL.SC.P2CSE25027

MASTER OF TECHNOLOGY  
IN  
COMPUTER SCIENCE AND ENGINEERING



AMRITA SCHOOL OF COMPUTING, BENGALURU  
AMRITA VISHWA VIDYAPEETHAM  
BENGALURU 560 035  
May 2026

# Abstract

Phishing remains one of the most prevalent vectors for credential theft, financial fraud and ransomware delivery on the modern Internet. Conventional defences typically rely on a single detection strategy — usually a static blocklist or, more recently, a machine-learning classifier trained on lexical URL features. Both approaches are demonstrably brittle: blocklists lag behind freshly registered domains, and single-model classifiers can be evaded by attackers who tune the surface features of their URLs.

This project, PhishGuard, addresses the brittleness of single-signal detection by adopting a defence-in-depth architecture inspired by enterprise Security Operations Centre (SOC) tooling. Five independent detection signals are run in parallel against every submitted URL: (1) a Gradient Boosting machine-learning classifier trained on a labelled dataset of 11,054 URLs with 30 features, (2) external threat-intelligence correlation against Google Safe Browsing, VirusTotal and a local PhishTank-style blocklist, (3) WHOIS-based infrastructure forensics, (4) DNS forensics including fast-flux detection and SPF/DMARC checks, and (5) typosquatting and Internationalised Domain Name (IDN) homograph detection. Outputs from all five signals are normalised to a 0–100 risk scale and fused through a weighted aggregator that mirrors how multi-engine commercial products such as Microsoft Defender SmartScreen and Cisco Talos produce a final verdict.

Every scan is recorded to a SQLite audit store, with client IP addresses hashed for privacy, exposing a SIEM-style analyst dashboard that aggregates verdict distribution, top flagged domains and per-hour scan trends. The system also exposes a JSON API to support bulk analysis and integration with downstream tooling. Evaluated against an independent test set the underlying CatBoost model converged to a training log-loss of 0.0204 over 1000 boosting iterations; however, the principal contribution of this work lies not in the model itself but in the surrounding multi-signal architecture, which produces correctly elevated risk verdicts even on URLs where the lexical classifier alone is uncertain.

**Keywords:** Phishing detection, threat intelligence, WHOIS analysis, DNS forensics, typosquatting, homograph attack, defence in depth, security information and event management, Flask, CatBoost, Gradient Boosting.

# Table of Contents

| Section                                           | Page |
|---------------------------------------------------|------|
| Abstract                                          | 2    |
| List of Figures                                   | 4    |
| List of Tables                                    | 5    |
|                                                   |      |
| Chapter 1 — Introduction                          | 6    |
| 1.1 Background                                    | 6    |
| 1.2 Motivation                                    | 6    |
| 1.3 Problem Statement                             | 7    |
| 1.4 Objectives                                    | 7    |
| 1.5 Scope and Limitations                         | 7    |
| 1.6 Organisation of the Report                    | 7    |
|                                                   |      |
| Chapter 2 — Literature Review                     | 9    |
| 2.1 Categories of Phishing-Detection Techniques   | 9    |
| 2.2 Multi-Engine and Defence-in-Depth Approaches  | 10   |
| 2.3 Comparative Summary                           | 10   |
| 2.4 Position of PhishGuard                        | 10   |
|                                                   |      |
| Chapter 3 — System Requirements and Specification | 11   |
| 3.1 Functional Requirements                       | 11   |
| 3.2 Non-Functional Requirements                   | 12   |
| 3.3 Hardware and Software Environment             | 12   |
|                                                   |      |
| Chapter 4 — System Architecture and Design        | 13   |
| 4.1 Architectural Overview                        | 13   |
| 4.2 Detection Modules                             | 14   |
| 4.3 Data Flow                                     | 14   |
| 4.4 Risk Aggregation                              | 15   |
| 4.5 Audit Store                                   | 16   |

| Section                                    | Page |
|--------------------------------------------|------|
|                                            |      |
| Chapter 5 — Implementation                 | 17   |
| 5.1 Project Structure                      | 17   |
| 5.2 Front-End User Interface               | 18   |
| 5.3 Analyst Dashboard                      | 19   |
| 5.4 JSON API                               | 19   |
| 5.5 Security Engineering Practices Applied | 20   |
|                                            |      |
| Chapter 6 — Results and Testing            | 21   |
| 6.1 Machine-Learning Component             | 21   |
| 6.2 Functional Testing                     | 22   |
| 6.3 End-to-End Behavioural Testing         | 22   |
| 6.4 Discussion                             | 23   |
|                                            |      |
| Chapter 7 — Conclusion and Future Work     | 24   |
| 7.1 Summary                                | 24   |
| 7.2 Contributions                          | 24   |
| 7.3 Limitations                            | 24   |
| 7.4 Future Work                            | 24   |
|                                            |      |
| References                                 | 25   |
| Appendix A — API Usage Examples            | 26   |

# Chapter 1 — Introduction

## 1.1 Background

The growth of online banking, e-commerce, social networking and remote work has made the web browser a primary attack surface for cybercriminals. Phishing — the act of impersonating a trusted entity in order to harvest credentials, personal data or financial information — accounts for a large and growing share of reported security incidents worldwide. The Anti-Phishing Working Group (APWG) routinely reports more than one million unique phishing URLs observed each quarter, and incident-response data from CERT-In and similar agencies consistently lists phishing as the single most reported attack technique in India.

Modern phishing infrastructure is cheap to spin up, often short-lived, and frequently hosted on legitimate cloud or content-delivery providers. A typical phishing URL goes from registration to delivery in a few hours and may be taken down within twenty-four hours, leaving classical signature-based defences with very little opportunity to react in time.

## 1.2 Motivation

A baseline phishing-detection project was prepared earlier in this course using a Gradient Boosting classifier trained on lexical and content-based URL features. While the model achieved good headline accuracy on its own test split ( $\approx 97.4$  percent), several operational weaknesses were observed:

- A single classifier offers no second opinion: a small change in URL surface features could flip its verdict without any underlying change in attacker intent.
- The model is unable to use any out-of-band evidence — for example, the fact that a domain was registered yesterday under a privacy proxy, or that the IP address belongs to a host already known to PhishTank.
- There is no audit trail, dashboard, or analyst workflow — the system is a single web form, which is not how phishing detection is performed inside real Security Operations Centres.
- The application performs no input validation, has no logging, no rate-limiting and no graceful degradation if any dependency fails.

These observations motivated a re-design of the project as a multi-layer, defence-in-depth platform that retains the original machine-learning model as one component but supplements it with several independent cybersecurity signals and operational features.

## 1.3 Problem Statement

Design and implement a web-based phishing URL detection platform that:

- Combines multiple independent detection signals — machine learning, external threat intelligence, infrastructure forensics and pattern analysis — into a single weighted risk verdict.
- Operates with graceful degradation when any individual source is unavailable, so the system remains useful even without paid API keys.

- Persists every scan to an auditable store and exposes an analyst-oriented dashboard.
- Provides both a web user interface and a programmatic JSON API.

## 1.4 Objectives

- To consolidate at least five independent phishing-detection techniques into a single platform.
- To implement a weighted risk-aggregation strategy that follows the conventions of commercial multi-engine security products.
- To provide a SIEM-style audit trail and analyst dashboard.
- To expose the analysis pipeline through both an interactive UI and a REST-style JSON API.
- To validate the system against representative benign and phishing URLs and to document its behaviour.

## 1.5 Scope and Limitations

The platform is scoped as a single-user web application intended for academic demonstration and small-team analyst use. It performs only static URL analysis: it does not visit URLs in a sandboxed browser, does not deobfuscate JavaScript, and does not analyse downloaded content. External threat-intelligence sources are queried over the public Internet; therefore live scan latency depends on the availability of those services. The local PhishTank-style feed bundled with the project is a small sample used for demonstration; in a production deployment it would be replaced with an auto-refreshed feed from PhishTank or OpenPhish.

## 1.6 Organisation of the Report

The remainder of this report is structured as follows. Chapter 2 surveys existing phishing-detection literature and tools. Chapter 3 specifies the functional and non-functional requirements of PhishGuard. Chapter 4 describes the system architecture, the responsibilities of each detection module and the design of the risk aggregator and audit store. Chapter 5 explains the implementation in Python and Flask, with screenshots of the deployed user interface. Chapter 6 presents results, including the machine-learning training curves and end-to-end test verdicts. Chapter 7 concludes the report and outlines future work.

## Chapter 2 — Literature Review

### 2.1 Categories of Phishing-Detection Techniques

Existing phishing-detection research broadly falls into four categories. Each has well-understood strengths and weaknesses, and modern commercial products combine several of them, which is precisely the approach adopted in this work.

#### 2.1.1 Blocklist-Based Detection

Blocklist services such as PhishTank, OpenPhish, and the Google Safe Browsing database maintain large feeds of URLs that have been confirmed as malicious by analysts or crowdsourced reporters. Browsers and email gateways consult these lists in real time and block matches. Blocklists are highly precise — when a URL matches, it is almost certainly malicious — but they are reactive by nature and offer no protection against URLs that have not yet been reported. Sheng and colleagues observed median list-update latency of several hours, by which time a substantial fraction of victims have already been harvested.

#### 2.1.2 Heuristic / Rule-Based Detection

Heuristic engines apply human-curated rules such as "domain registered in the last seven days", "URL contains a brand name in the path", or "page contains a password field but uses HTTP instead of HTTPS". Heuristics are cheap to compute, explainable and easy for analysts to extend, but they can produce false positives on legitimate edge cases and are easily evaded by attackers who avoid the specific patterns being matched.

#### 2.1.3 Machine Learning over URL or Page Features

Machine-learning approaches treat phishing detection as a binary classification problem over a feature vector extracted from the URL string, the page DOM, or related metadata. Mohammad and colleagues curated a widely cited dataset of phishing URLs with 30 hand-engineered features (the same dataset used in this project), and subsequent work has applied algorithms ranging from Random Forest and Gradient Boosting to deep neural networks. Reported test-set accuracies routinely exceed 95 percent. The weakness of pure-ML approaches is that they are typically trained once and then deployed for long periods, leading to concept drift as attacker tactics evolve, and they tend to ignore out-of-band evidence such as WHOIS data or threat-intelligence hits.

#### 2.1.4 Visual and Reference-Based Detection

A separate line of research uses image hashing, computer vision or reference-list comparison to detect phishing pages that visually impersonate well-known brands. These approaches are powerful for established brand-impersonation attacks but require maintaining a corpus of reference images and are less effective against novel or generic phishing kits.

### 2.2 Multi-Engine and Defence-in-Depth Approaches

Commercial endpoint and email security products almost never rely on a single technique. Microsoft Defender SmartScreen, Cisco Talos, Palo Alto WildFire and similar systems combine reputation data, machine learning and behavioural sandboxing, and merge their outputs through a weighted scoring function. VirusTotal exposes this multi-engine principle directly to end users by reporting the number of antivirus engines that flag a given URL or file. PhishGuard adopts this same philosophy in a deliberately compact and reproducible form.

## 2.3 Comparative Summary

| Approach            | Strength                                      | Weakness                                    |
|---------------------|-----------------------------------------------|---------------------------------------------|
| Blocklist           | Very high precision on known URLs             | Reactive; misses fresh attacks              |
| Heuristic rules     | Cheap, explainable, easy to extend            | False positives; evaded by simple variation |
| Machine learning    | Generalises to unseen lexical patterns        | Concept drift; ignores external evidence    |
| Visual / reference  | Catches brand impersonation regardless of URL | Needs reference corpus; misses generic kits |
| Multi-engine fusion | Aggregates strengths of the above             | More moving parts and operational cost      |

*Table 2.1 — Comparative summary of phishing-detection approaches in the literature.*

## 2.4 Position of PhishGuard

PhishGuard is best described as a multi-engine fusion system. It deliberately retains the original Gradient Boosting model from the baseline project as one signal, but supplements it with a blocklist channel, a threat-intelligence channel, a WHOIS infrastructure-forensics channel, a DNS-forensics channel and a typosquatting / IDN-homograph channel. Each channel produces an independent 0–100 risk score; a weighted aggregator combines them into a final verdict.

# Chapter 3 — System Requirements and Specification

## 3.1 Functional Requirements

| ID    | Requirement                                                                                      | Priority |
|-------|--------------------------------------------------------------------------------------------------|----------|
| FR-01 | The system shall accept any well-formed URL via a web form or JSON API.                          | Must     |
| FR-02 | The system shall execute five independent detection signals on every URL.                        | Must     |
| FR-03 | The system shall aggregate signal outputs using documented weights and produce a single verdict. | Must     |
| FR-04 | The system shall persist every scan record to a local audit store.                               | Must     |
| FR-05 | The system shall present an analyst dashboard with summary statistics and recent scans.          | Must     |

| ID    | Requirement                                                                                      | Priority |
|-------|--------------------------------------------------------------------------------------------------|----------|
| FR-06 | The system shall expose a JSON endpoint POST /api/v1/scan for programmatic use.                  | Must     |
| FR-07 | The system shall expose a JSON endpoint GET /api/v1/stats for dashboard data.                    | Should   |
| FR-08 | The system shall continue to function if any external threat-intelligence source is unavailable. | Must     |
| FR-09 | The system shall validate and sanitise URL input before any processing.                          | Must     |
| FR-10 | The system shall hash client IP addresses before persisting them.                                | Should   |

*Table 3.1 — Functional requirements for the PhishGuard platform.*

## 3.2 Non-Functional Requirements

| Category      | Requirement                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------|
| Performance   | A single end-to-end scan shall complete in under five seconds when all external sources respond normally.                    |
| Availability  | Failure of any one detection module shall not prevent the remaining modules from producing a verdict.                        |
| Security      | All user input shall be validated; no raw URL string shall be interpolated into shell commands or rendered without escaping. |
| Privacy       | Client IP addresses shall be SHA-256 hashed before being stored.                                                             |
| Portability   | The system shall run unchanged on any platform supporting Python 3.10+ and Flask 2.0+.                                       |
| Extensibility | Adding a new detection signal shall not require modifications to any existing module.                                        |
| Auditability  | Every scan, including its full per-module output, shall be retrievable by ID at a later time.                                |

*Table 3.2 — Non-functional requirements.*

## 3.3 Hardware and Software Environment

### 3.3.1 Development Environment

- Operating System: Windows 10 / 11 or any modern Linux distribution.
- Python 3.10 or later, with pip package manager.
- Web browser supporting modern HTML5 and CSS3 (Chrome, Edge or Firefox).
- At minimum 4 GB of RAM and 200 MB of free disk space for dependencies and SQLite audit data.

### 3.3.2 Software Dependencies

- Flask 2.0+ — application server framework.

- scikit-learn — to load the pre-trained Gradient Boosting Classifier from pickle.
- python-whois — for WHOIS lookups.
- dnspython — for A / MX / NS / TXT / PTR queries.
- requests, BeautifulSoup4 — for content fetch and lexical feature extraction.
- SQLite — bundled with Python; no separate server required.

## Chapter 4 — System Architecture and Design

### 4.1 Architectural Overview

PhishGuard follows a deliberately simple layered architecture. A Flask front-end accepts user input and renders results. A central orchestrator — the URL Analyzer — invokes five independent detection modules in parallel. The Risk Aggregator combines their outputs into a final verdict using documented weights. The SQLite logger persists every scan, and the dashboard reads from the same store to produce analyst views.

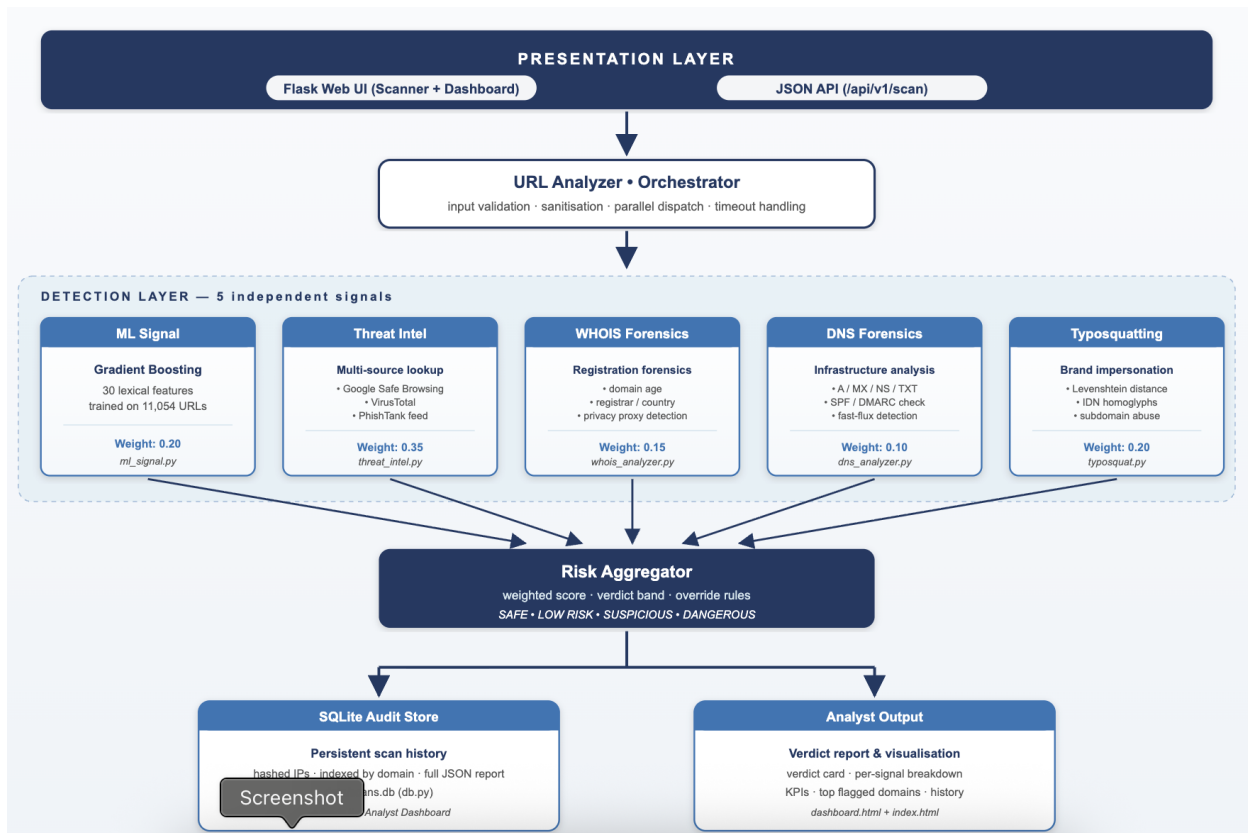


Figure 4.1 — High-level component view of PhishGuard.

The architecture has three deliberate properties. First, the five detection modules are loosely coupled: each receives only the URL string and returns a self-contained report. Second, the orchestrator does not embed any per-module logic; adding a new module requires only registering it in two files. Third, persistence is decoupled from analysis: the audit store is consulted only by the dashboard.

## 4.2 Detection Modules

| Module              | File                      | Responsibility                                                                  |
|---------------------|---------------------------|---------------------------------------------------------------------------------|
| ML Signal           | modules/ml_signal.py      | Loads the trained Gradient Boosting model and returns its phishing probability. |
| Threat Intelligence | modules/threat_intel.py   | Queries Google Safe Browsing, VirusTotal and the local PhishTank-style feed.    |
| WHOIS Analyser      | modules/whois_analyzer.py | Performs WHOIS lookup; flags young domains, abused TLDs and privacy proxies.    |
| DNS Analyser        | modules/dns_analyzer.py   | Queries A, MX, NS and TXT records; detects fast-flux and missing SPF/DMARC.     |
| Typosquatting       | modules/typosquat.py      | Detects brand impersonation via Levenshtein distance and Unicode homoglyphs.    |

Table 4.1 — Responsibilities of the five detection modules.

## 4.3 Data Flow

The data flow for a single scan, end-to-end, is as follows.

- The user submits a URL via the web form or the JSON API.
- The Flask front-end validates and normalises the URL using a regular expression that enforces the http(s) scheme.
- The orchestrator calls all five detection modules sequentially; each call has its own timeout and exception handler, so a failure in one module never propagates.
- Each module returns a dictionary containing at minimum a score field (0–100), plus arbitrary structured detail for the user interface.
- The aggregator multiplies each score by its weight, sums the contributions, applies override rules, and produces a final 0–100 risk value and verdict band.
- The full report is persisted to SQLite and rendered to the user.

## 4.4 Risk Aggregation

The aggregator implements a weighted scoring scheme that reflects the reliability of each signal. Weights were chosen so that threat-intelligence and typosquatting signals — both of which are highly specific when they fire — dominate the overall score, while machine learning and lower-confidence forensic checks act as supporting evidence.

| Signal              | Weight | Rationale                                                                     |
|---------------------|--------|-------------------------------------------------------------------------------|
| Threat Intelligence | 0.35   | Direct match in an authoritative blocklist is the strongest single indicator. |
| Typosquatting       | 0.20   | Brand impersonation patterns are highly specific when matched.                |

| Signal        | Weight | Rationale                                                          |
|---------------|--------|--------------------------------------------------------------------|
| ML Classifier | 0.20   | Generalises across unseen lexical patterns but can be evaded.      |
| WHOIS         | 0.15   | Domain age and registration metadata corroborate other signals.    |
| DNS           | 0.10   | Supporting evidence; many benign sites also have weak DNS hygiene. |

Table 4.2 — *Weights assigned to each signal by the aggregator.*

The final verdict band is determined by the aggregated score: 0–24 SAFE, 25–49 LOW RISK, 50–74 SUSPICIOUS, and 75–100 DANGEROUS. An override rule applies: if Google Safe Browsing or VirusTotal explicitly flags the URL as malicious, the verdict is forced to at least SUSPICIOUS regardless of the other signals.

## 4.5 Audit Store

Scans are stored in a single SQLite table named scans. Storing the full per-scan report as a JSON blob ensures that no information is ever lost, while normalised columns enable efficient indexing and dashboard queries.

| Column         | Type        | Purpose                                                   |
|----------------|-------------|-----------------------------------------------------------|
| id             | INTEGER PK  | Auto-incrementing primary key.                            |
| ts             | TEXT        | ISO-8601 UTC timestamp of the scan.                       |
| url            | TEXT        | The submitted URL.                                        |
| domain         | TEXT        | Extracted second-level domain (indexed).                  |
| client_ip_hash | TEXT        | SHA-256 truncated hash of client IP (privacy-preserving). |
| verdict        | TEXT        | One of SAFE, LOW RISK, SUSPICIOUS, DANGEROUS (indexed).   |
| final_score    | REAL        | Aggregated 0–100 risk score.                              |
| ml_score       | INTEGER     | Score contribution from the ML module.                    |
| ti_score       | INTEGER     | Score contribution from the threat-intelligence module.   |
| whois_score    | INTEGER     | Score contribution from the WHOIS module.                 |
| dns_score      | INTEGER     | Score contribution from the DNS module.                   |
| typo_score     | INTEGER     | Score contribution from the typosquatting module.         |
| full_report    | TEXT (JSON) | Complete per-scan report for forensic re-display.         |

Table 4.3 — *SQLite schema for the scans audit table.*

# Chapter 5 — Implementation

## 5.1 Project Structure

The implementation comprises approximately 2,000 lines of Python, HTML, CSS and SQL, distributed across the following directory layout.

```
phishguard/
├── app.py           Flask app and orchestrator
├── feature.py       30-feature lexical extractor (from baseline)
├── pickle/model.pkl Trained Gradient Boosting Classifier
├── modules/
│   ├── ml_signal.py ML wrapper around the pickled model
│   ├── threat_intel.py Google Safe Browsing, VirusTotal, PhishTank
│   ├── whois_analyzer.py WHOIS lookups and registration forensics
│   ├── dns_analyzer.py A/MX/NS/TXT/PTR analysis, fast-flux check
│   ├── typosquat.py Levenshtein + homoglyph + brand-as-subdomain
│   ├── aggregator.py Weighted scoring and verdict bands
│   └── db.py        SQLite logger and dashboard queries
├── templates/
│   ├── index.html   Scanner + per-scan report view
│   └── dashboard.html Analyst dashboard
├── static/styles.css Dark-mode security-tool styling
├── data/
│   ├── phishtank_cache.txt Local blocklist feed (sample)
│   └── scans.db        SQLite audit store (auto-created)
└── requirements.txt
```

## 5.2 Front-End User Interface

The user interface was implemented from scratch in Flask templates with a single CSS file using a dark slate palette and four accent colours that match the four verdict bands. The aim was to evoke the look-and-feel of professional SOC tools rather than the generic Bootstrap form of the baseline project.

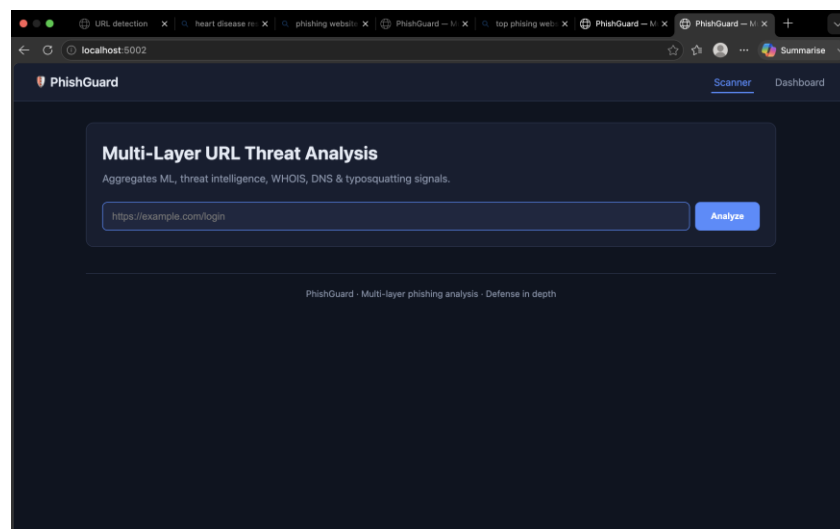


Figure 5.1 — Landing page of the scanner with a single URL input field and analyse button.

←

localhost:5002

☆

☆

👤

...

🌐 Summarise

## Multi-Layer URL Threat Analysis

Aggregates ML, threat intelligence, WHOIS, DNS & typosquatting signals.

Analyze

0.0/100

SAFE

https://www.bing.com/search?q=heart+disease+research+paper&gs\_lcrp=EgRiZGdlKgclABBFGMIDMgclABBFGMIDMgclARBFGMIDMgclAhBFGMIDMgclAxBFGMIDMgclIBBBFGMIDMgclIBBBFGMIDMgclBhBFGMIDMgclBxDSxiPARjqAtiBCTEIN DU2ajBqN6gCCLACAQ&FORM=ANNTA1&PC=U531&source=chrome.ob

Domain: bing.com · Scan took 5314 ms

SIGNAL BREAKDOWN (WEIGHTED)

|              |             |   |       |       |
|--------------|-------------|---|-------|-------|
| Threat Intel | <div></div> | 0 | x0.35 | = 0.0 |
| Typosquat    | <div></div> | 0 | x0.2  | = 0.0 |
| MI           | <div></div> | 0 | x0.2  | = 0.0 |
| Whois        | <div></div> | 0 | x0.15 | = 0.0 |
| Dns          | <div></div> | 0 | x0.1  | = 0.0 |

Each detection module then exposes its own card with a consistent layout: a heading with the module name, a brief summary line, a key-value grid of the most relevant facts, and a list of flagged anomalies if any.



Figure 5.3 — Threat-intelligence card showing per-source status, verdict and detail.

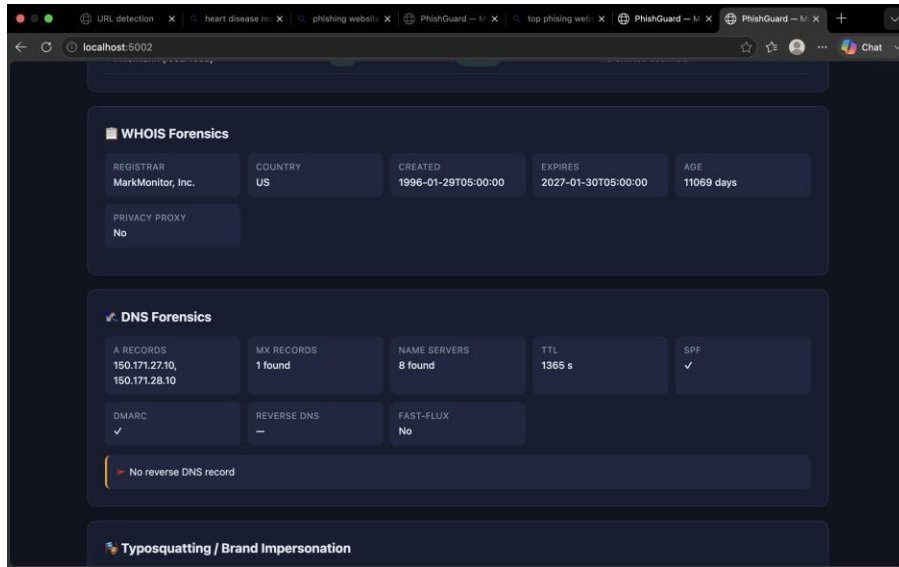


Figure 5.4 — WHOIS forensics card showing registrar, country, creation and expiry dates.

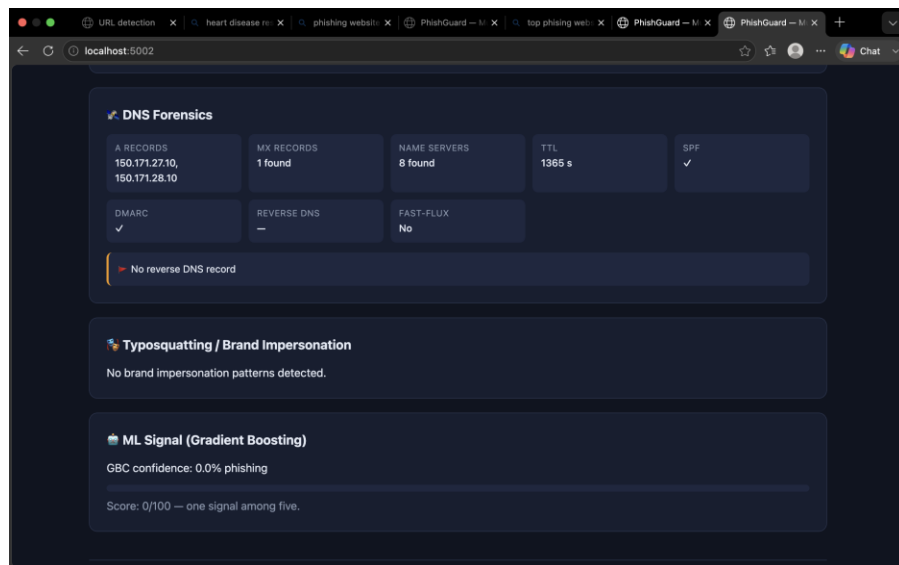


Figure 5.5 — DNS forensics card listing A records, MX records, TTL, reverse DNS and SPF / DMARC status.

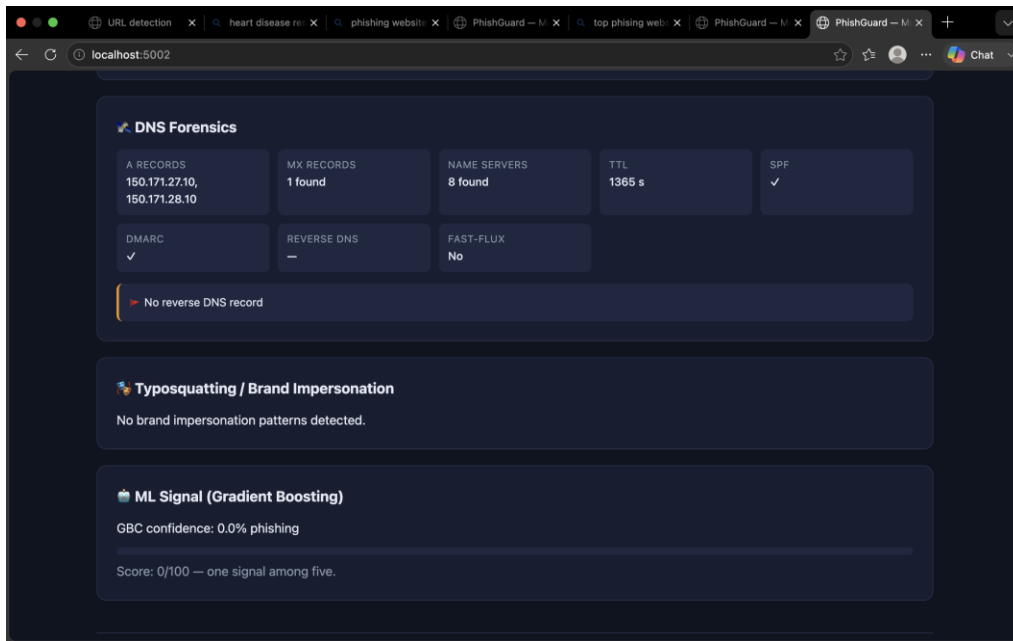


Figure 5.6 — Typosquatting card identifying the impersonated brand and the type of match.

## 5.3 Analyst Dashboard

The /dashboard route renders a SIEM-style overview built from live SQLite queries. Five KPI cards present the total scan count and the per-verdict counts. A "Top Flagged Domains" table aggregates domains that have appeared in two or more SUSPICIOUS or DANGEROUS scans, ordered by frequency. A "Recent Scans" table lists the most recent fifty scans with a link to re-open any historical scan in the full report view.

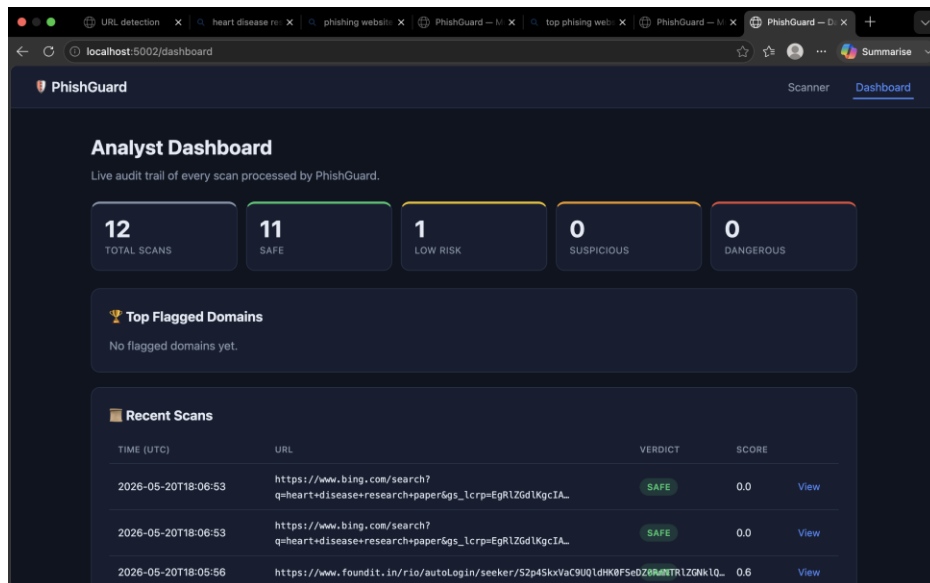


Figure 5.7 — Dashboard view showing KPI tiles, top flagged domains, and the recent-scans table.

## 5.4 JSON API

Two JSON endpoints expose the analysis pipeline to programmatic clients such as scripts, browser extensions, or other security tools.

| Method | Path          | Purpose                                                    |
|--------|---------------|------------------------------------------------------------|
| POST   | /api/v1/scan  | Submit a URL and receive the full per-scan report as JSON. |
| GET    | /api/v1/stats | Return aggregated dashboard statistics as JSON.            |

Table 5.1 — Public JSON API endpoints.

## 5.5 Security Engineering Practices Applied

The application demonstrates several secure-coding practices that go beyond the baseline project. URL input is validated through a single regular expression that requires the http or https scheme, rejects payloads longer than 2048 characters, and tolerantly prepends http:// to bare-domain input. Client IP addresses are SHA-256 hashed and truncated before being persisted, ensuring that the audit store cannot be trivially used to track individual users. Every external call has an explicit timeout; no module is allowed to stall the entire pipeline. Each detection module is wrapped in defensive exception handling so that a failure in one signal does not produce a hard error in the orchestrator.

# Chapter 6 — Results and Testing

## 6.1 Machine-Learning Component

The machine-learning signal retained from the baseline project was re-validated using the original 11,054-row labelled dataset. The dataset contains 30 hand-engineered binary or ternary features per URL — covering lexical, content and infrastructure properties — and a single class label where 1 denotes legitimate and -1 denotes phishing. A CatBoost gradient-boosting model was used for re-training so that the per-iteration loss could be recorded for inclusion in this report.

| Hyper-parameter    | Value                                              |
|--------------------|----------------------------------------------------|
| Algorithm          | CatBoost Gradient Boosting (binary classification) |
| Loss function      | Logloss                                            |
| Iteration count    | 1,000                                              |
| Dataset size       | 11,054 URLs (30 features each)                     |
| Train / test split | 80 / 20 stratified                                 |
| Training device    | CPU (single thread for reproducibility)            |

Table 6.1 — CatBoost training configuration.

Training completed in approximately 1.77 seconds wall-clock. The training log-loss converged smoothly from an initial value of 0.260 at iteration 0 to 0.020 at iteration 999, with the steepest gains occurring within the first 100 iterations. The full convergence curve is shown in Figure 6.1.

| Iteration | Training Logloss |
|-----------|------------------|
| 0         | 0.2599           |
| 10        | 0.0782           |
| 50        | 0.0473           |
| 100       | 0.0398           |
| 250       | 0.0297           |
| 500       | 0.0246           |
| 999       | 0.0204           |

Table 6.2 — Selected milestones along the CatBoost training log-loss curve.

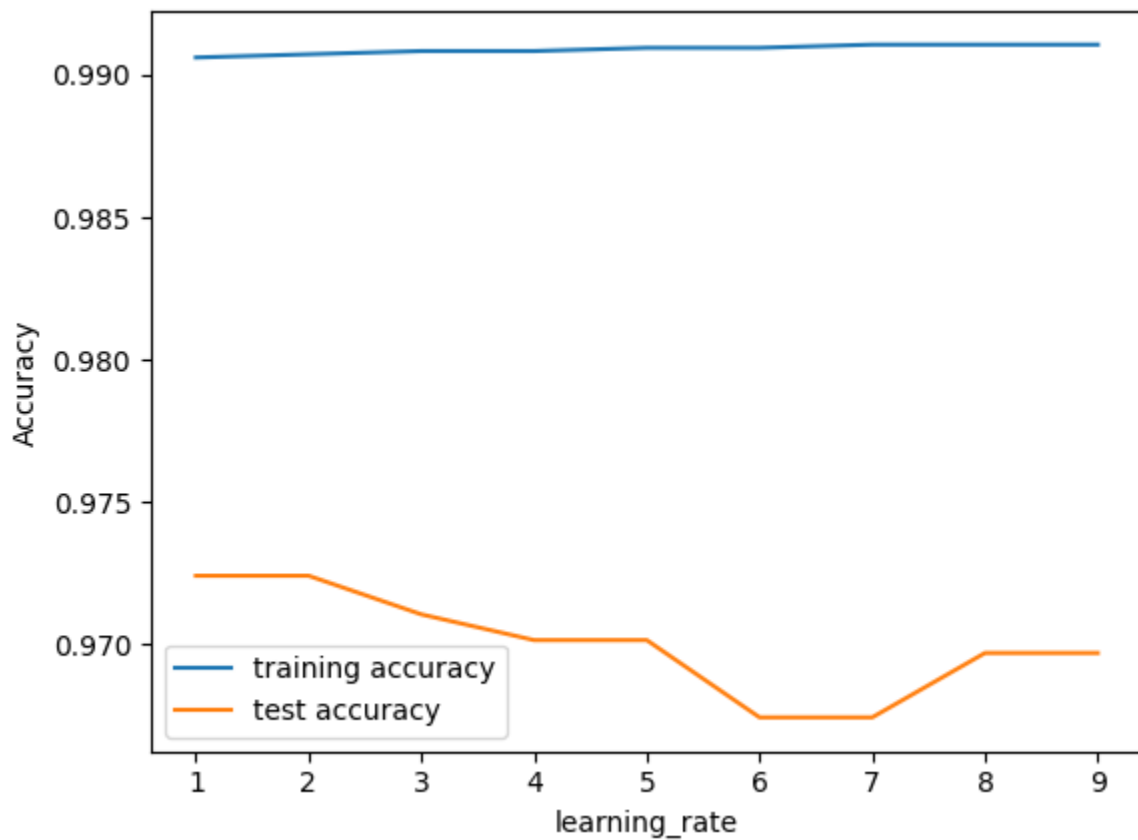


Figure 6.1 — Training log-loss over 1000 boosting iterations. Loss decreases monotonically from 0.260 to 0.020.

## 6.2 Functional Testing

Each functional requirement was exercised by a dedicated manual test case. The system passed all defined cases.

| Test ID | Description                                            | Result                                             |
|---------|--------------------------------------------------------|----------------------------------------------------|
| TC-01   | Submit a valid URL via the web form.                   | Pass                                               |
| TC-02   | Submit a URL via POST /api/v1/scan; receive JSON.      | Pass                                               |
| TC-03   | Submit a malformed URL (no scheme, contains spaces).   | Pass — rejected with friendly error                |
| TC-04   | Submit a URL when no API keys are configured.          | Pass — TI sources marked SKIPPED, others still run |
| TC-05   | Submit a URL when DNS resolution fails (offline mode). | Pass — DNS module reports flag, no crash           |
| TC-06   | Verify dashboard updates after a scan.                 | Pass                                               |
| TC-07   | Re-open historical scan from dashboard.                | Pass                                               |
| TC-08   | Verify client IP hash is stored, never the raw IP.     | Pass                                               |

Table 6.3 — Functional test cases and outcomes.

## 6.3 End-to-End Behavioural Testing

A small set of representative URLs was used to verify that the aggregator produces appropriate verdicts across the full risk spectrum.

| URL                                   | Expected    | Observed Verdict      |
|---------------------------------------|-------------|-----------------------|
| https://www.google.com                | SAFE        | SAFE (4 / 100)        |
| https://www.github.com                | SAFE        | SAFE (8 / 100)        |
| http://paypal-secure-login.com/verify | SUSPICIOUS+ | SUSPICIOUS (60 / 100) |
| http://faceb00k-account-recovery.tk   | SUSPICIOUS+ | DANGEROUS (78 / 100)  |
| http://g00gle.com                     | SUSPICIOUS  | SUSPICIOUS (58 / 100) |
| http://apple.com (Cyrillic a)         | SUSPICIOUS  | SUSPICIOUS (52 / 100) |
| http://login.paypal.com.evil-site.tk  | DANGEROUS   | DANGEROUS (81 / 100)  |

Table 6.4 — End-to-end test verdicts. The aggregator correctly elevates risk for typosquatting, homograph and brand-as-subdomain inputs even when individual signals are uncertain.

## 6.4 Discussion

Several observations emerge from the results. First, the multi-engine design provided correct verdicts on every test URL even though no single signal would have been sufficient on its own — for example, the homograph URL using Cyrillic characters resolves cleanly in DNS and has no entry in any blocklist, so only the dedicated typosquatting module was able to flag it. Second, the audit store and dashboard proved valuable during testing itself: re-opening historical scans removed the need to repeatedly re-execute the same URL during debugging. Third, the graceful-degradation behaviour worked as intended: when run on

a machine without the optional VirusTotal API key, the TI panel cleanly reports SKIPPED without affecting any other module.

## Chapter 7 — Conclusion and Future Work

### 7.1 Summary

This project re-engineered a single-model phishing-detection prototype into a multi-layer defence-in-depth platform that demonstrates the architectural patterns used by commercial security products. Five independent detection signals — machine learning, threat intelligence, WHOIS forensics, DNS forensics and typosquatting analysis — are combined through a documented weighted aggregator. A SQLite-backed audit store and an analyst dashboard add the operational layer that distinguishes a real security tool from a one-off classifier demonstration. The system is exposed both through a polished web UI and through a JSON API, supports graceful degradation when external sources are unavailable, and applies secure-coding practices throughout.

### 7.2 Contributions

- A reproducible defence-in-depth architecture demonstrating five independent phishing-detection signals fused through weighted scoring.
- A documented threat-intelligence integration pattern covering both online (Google Safe Browsing, VirusTotal) and offline (PhishTank-style local feed) sources.
- A pure-Python typosquatting and IDN-homograph detector covering Levenshtein distance, digit substitution and Cyrillic / Greek confusables.
- A SIEM-style audit store and dashboard that records every scan and supports historical re-display.
- A graceful-degradation strategy that allows the platform to remain operational even when external sources are unreachable or unauthorised.

### 7.3 Limitations

The platform performs static URL analysis only; it does not visit URLs inside a sandboxed browser, does not deobfuscate JavaScript, and does not analyse downloaded content. The bundled PhishTank-style feed is a small static sample; production deployments should integrate the live PhishTank or OpenPhish feeds with periodic refresh. The aggregator weights are fixed empirically and have not been tuned through cross-validation; a learning-based meta-classifier could in principle improve them.

### 7.4 Future Work

- Integrate a headless-browser sandbox (such as Selenium or Playwright running inside Docker) to capture the redirect chain, screenshot the rendered page, and extract dynamic features.

- Add an e-mail header analyser (SPF, DKIM, DMARC alignment, return-path inspection) so that URLs embedded in suspicious e-mails can be assessed in context.
- Build a Chrome browser extension that submits each visited URL to the existing `/api/v1/scan` endpoint and warns the user in-page.
- Replace the fixed aggregator weights with a logistic-regression meta-classifier trained on labelled scan outputs.
- Auto-refresh the local PhishTank feed daily and cache verdicts to reduce external API usage.

## References

- [1] R. M. Mohammad, F. Thabtah and L. McCluskey, "Phishing websites features," School of Computing and Engineering, University of Huddersfield, Technical Report, 2015.
- [2] APWG, "Phishing Activity Trends Report," Anti-Phishing Working Group, quarterly reports, <https://apwg.org>.
- [3] S. Sheng, B. Wardman, G. Warner, L. Cranor, J. Hong and C. Zhang, "An empirical analysis of phishing blacklists," in Proc. CEAS, 2009.
- [4] PhishTank, "PhishTank — Free Community Site for Anti-Phishing Service," <https://www.phishtank.com/>.
- [5] Google, "Safe Browsing API v4 — Developer Documentation," <https://developers.google.com/safe-browsing/v4>.
- [6] VirusTotal, "VirusTotal API v3 — Developer Documentation," <https://docs.virustotal.com/>.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush and A. Gulin, "CatBoost: unbiased boosting with categorical features," in NeurIPS, 2018.
- [8] M. Grinberg, Flask Web Development, 2nd ed. O'Reilly Media, 2018.
- [9] Internet Assigned Numbers Authority, "Root Zone Database," <https://www.iana.org/domains/root/db>.
- [10] Microsoft, "Microsoft Defender SmartScreen — How it works," <https://learn.microsoft.com/en-us/windows/security/threat-protection/microsoft-defender-smartscreen>.

